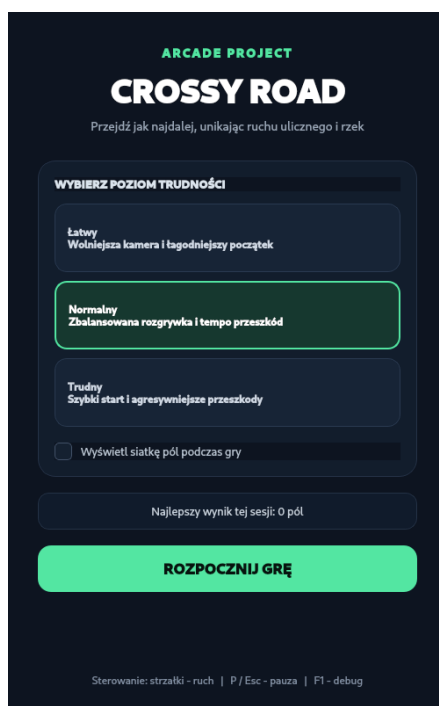


Sprawozdanie z realizacji aplikacji

Crossy Road Deluxe - gra zręcznościowa z rozbudowanym interfejsem GUI

Przedmiot	Architektura systemów komputerowych
Autorzy	Łukasz Orluk, nr albumu 197641 Łukasz Mroczek, nr albumu 198146
Temat	Aplikacja okienkowa w języku Python z wykorzystaniem PyQt5
Rok akademicki	2025/2026



Rys. 1. Nowy ekran startowy aplikacji umożliwiający wybór poziomu trudności.

Spis treści

1.	Sformułowanie zadania i cel projektu
2.	Założenia szczegółowe
3.	Technologie oraz struktura aplikacji
4.	Opis przyjętych rozwiązań programowych
5.	Obsługa i uruchomienie aplikacji
6.	Weryfikacja działania i osiągnięte wyniki
7.	Dyskusja wyników - zalety, wady i możliwości rozwoju
8.	Podsumowanie

Niniejsze sprawozdanie opisuje rozwiązania rzeczywiście obecne w kodzie projektu. Przytoczone fragmenty kodu zostały ograniczone do kluczowych mechanizmów, zgodnie z wymaganiem, aby nie umieszczać wydruków całych programów.

1. Sformułowanie zadania i cel projektu

Celem projektu było opracowanie kompletnej aplikacji okienkowej będącej uproszczoną wersją gry typu Crossy Road. Zadaniem użytkownika jest przemieszczanie postaci po planszy złożonej z pasów terenu, omijanie pojazdów, korzystanie z unoszących się kłód podczas przeprawy przez rzekę oraz unikanie pozostania poza obszarem widocznym kamery.

Program został zrealizowany w języku Python z wykorzystaniem biblioteki PyQt5. Po rozbudowie aplikacja posiada graficzny ekran startowy, wybór jednego z trzech poziomów trudności, opcję wyświetlenia siatki, panel HUD podczas rozgrywki, pauzę oraz ekran końcowy z możliwością ponownej gry lub powrotu do menu. Rdzeń gry nadal generuje świat dynamicznie, obsługuje kolizje, zapis, odczyt i odtworzenie ruchów gracza.

Zakres funkcjonalny projektu obejmuje:

- renderowanie planszy o wymiarach 500 x 800 pikseli, podzielonej na pola o boku 50 pikseli;
- trzy rodzaje pasów terenu: bezpieczny teren trawiasty, jezdnię oraz rzekę;
- ruchome przeszkody i elementy pomocnicze: samochody, kłody i drzewa;
- wykrywanie kolizji, warunków przegranej oraz blokowania pól przez drzewa;
- stopniowe zwiększanie poziomu trudności wraz z postępem gracza;
- zapis i odczyt bieżącego stanu rozgrywki do pliku JSON;

- mechanizm replay oraz prosty tryb diagnostyczny uruchamiany klawiszem F1;
- interfejs GUI z menu, wyborem trudności, panelem HUD, pauzą i ekranem końca gry.

2. Założenia szczegółowe

W celu jednoznacznego określenia działania aplikacji przyjęto następujące założenia projektowe:

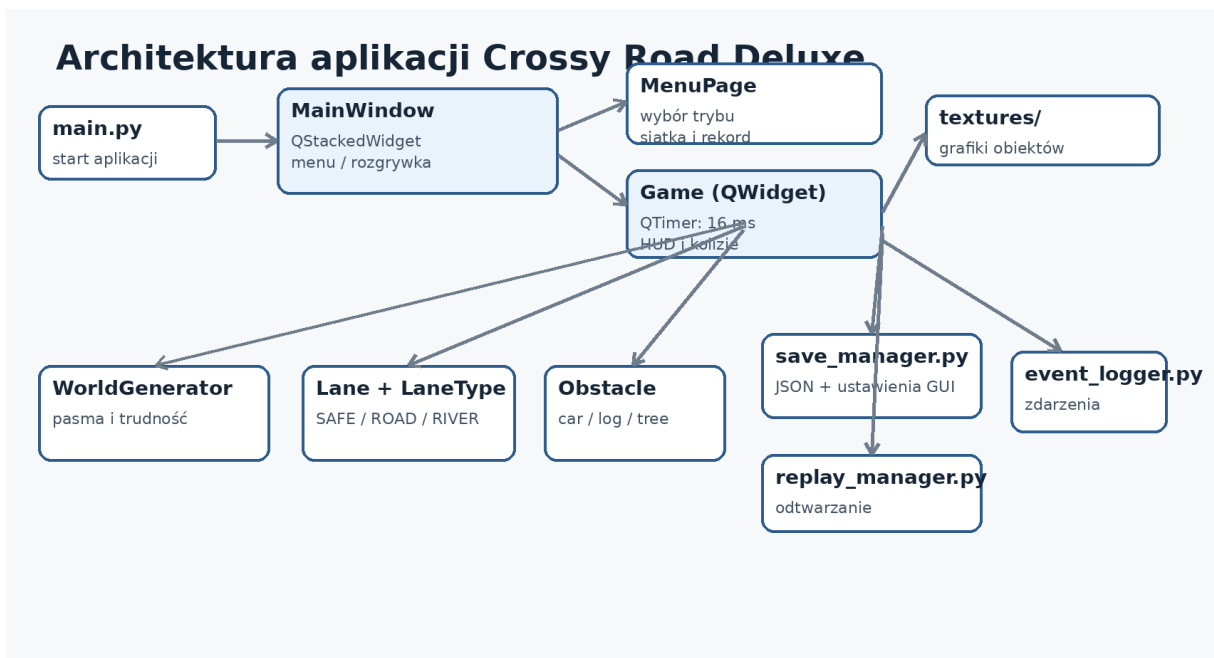
Obszar	Przyjęte założenie
Model planszy	Świat jest zbiorem wierszy (pasów), a pozycja gracza jest opisana numerem kolumny i numerem wiersza świata.
Sterowanie	Gracz porusza się o jedno pole strzałkami: góra, dół, lewo, prawo.
Warunek bezpieczeństwa	Na trawie drzewa blokują przejście; na drodze zderzenie z samochodem kończy grę; w rzece gracz musi znajdować się na kłodzie.
Dynamika gry	Stan gry jest odświeżany przez timer co 16 ms, co odpowiada płynnej animacji około 60 klatek na sekundę.
Trudność	Po każdych pięciu przebytych wierszach trudność rośnie, zwiększając prędkość obiektów oraz częstotliwość ich pojawiania się.
Dane trwałe	Stan planszy i postaci może być zapisany do pliku save.json i później odtworzony.
Grafika	Elementy są rysowane przy użyciu tekstur PNG, a w przypadku braku tekstury przewidziano podstawowe kolory zastępcze.
Interfejs GUI	Przed rozpoczęciem gry użytkownik wybiera tryb Łatwy, Normalny albo Trudny oraz opcjonalne wyświetlenie siatki pól.
Sesja gry	HUD prezentuje wynik i tryb, a pauza oraz ekran końcowy umożliwiają kontrolę przebiegu rozgrywki bez zamykania aplikacji.

Istotnym założeniem generatora jest zachowanie grywalności planszy: po pasie rzeki nie jest generowany bezpośrednio kolejny pas rzeki, a na pasach bezpiecznych utrzymywana jest co najmniej jedna potencjalnie dostępna kolumna przejścia.

3. Technologie oraz struktura aplikacji

Aplikacja ma budowę modułową. Nowy moduł `app_window.py` odpowiada za główne okno programu, ekran menu oraz przełączanie pomiędzy menu i planszą gry. Klasa `Game` pełni rolę kontrolera właściwej rozgrywki: steruje zegarem gry, HUD-em, stanami pauzy i końca gry, przetwarza zdarzenia oraz rysuje scenę. Pozostałe moduły realizują wydzielone odpowiedzialności związane z generatorem, obiektami i zapisem danych.

Plik / moduł	Odpowiedzialność
<code>main.py</code>	Utworzenie <code>QApplication</code> oraz uruchomienie obiektu <code>MainWindow</code> .
<code>app_window.py</code>	Menu startowe, presety trudności, opcja siatki, rekord sesji i przełączanie ekranów.
<code>config.py</code>	Stałe wymiarów okna, rozmiaru pola i liczby kolumn/wierszy widocznych.
<code>lane_types.py</code>	Typ wyliczeniowy <code>LaneType</code> : <code>SAFE</code> , <code>ROAD</code> , <code>RIVER</code> .
<code>lane.py</code>	Model pojedynczego pasa wraz z parametrami ruchu i listą obiektów.
<code>obstacle.py</code>	Model obiektu na pasie i aktualizacja jego położenia.
<code>generator.py</code>	Proceduralne tworzenie pasów zależne od poziomu trudności.
<code>game.py</code>	Pętla gry, renderowanie, kolizje, kamera, HUD, pauza, ekran końcowy i integracja modułów.
<code>save_manager.py</code>	Serializacja i deserializacja stanu gry do formatu <code>JSON</code> .
<code>replay_manager.py</code>	Zapis kolejności ruchów oraz ich ponowne odtworzenie.
<code>event_logger.py</code>	Wypisywanie podstawowych zdarzeń aplikacji w konsoli.



Rys. 2. Podział programu na moduły oraz główne zależności pomiędzy nimi.

4. Opis przyjętych rozwiązań programowych

4.1. Reprezentacja świata i typów pasów

Podstawową jednostką planszy jest obiekt klasy Lane. Każdy pas posiada typ, kierunek poruszania obiektów, prędkość, interwał generowania nowych przeszkód oraz listę aktualnie istniejących obiektów. Typ pasa jest reprezentowany za pomocą typu wyliczeniowego, co ogranicza ryzyko użycia niepoprawnej wartości tekstowej.

Fragment 1. Typy pasów i podstawowa struktura pasa:

```

class LaneType(Enum):
    SAFE = auto()
    ROAD = auto()
    RIVER = auto()

class Lane:
    def __init__(self, index, lane_type, direction=1, speed=0, spawn_interval=100):
        self.index = index
        self.lane_type = lane_type
        self.direction = direction
        self.speed = speed
        self.spawn_interval = spawn_interval
        self.obstacles = []
  
```

4.2. Proceduralne generowanie planszy

Moduł WorldGenerator generuje nowe pasy wtedy, gdy są potrzebne do narysowania obszaru przed graczem i wokół kamery. Losowanie jest ważne: wraz ze wzrostem trudności zmniejsza się udział bezpiecznej trawy, a rośnie prawdopodobieństwo jezdni i rzek. Ponadto generator usuwa możliwość wylosowania rzeki bezpośrednio po innej rzece.

Fragment 2. Ograniczenie dwóch rzek z rzędu i skalowanie parametrów:

```

if self.last_lane_type == LaneType.RIVER:
    possible.remove(LaneType.RIVER)

if lane_type == LaneType.ROAD:
    speed = 2.0 + difficulty * 0.6
    spawn_interval = max(35, int(120 - difficulty * 12))
else:
    speed = 1.5 + difficulty * 0.45
    spawn_interval = max(45, int(140 - difficulty * 10))

```

Na pasach typu SAFE dodawane są drzewa. Algorytm pomija jedną aktualizowaną kolumnę `safe_path_col`, dzięki czemu projekt przewiduje korytarz przejścia i ogranicza sytuacje całkowitego zablokowania dalszej drogi.

4.3. Pętla gry, kamera oraz trudność

Aktualizacja aplikacji odbywa się cyklicznie przez obiekt `QTimer` uruchamiany z okresem 16 ms. W każdym kroku sprawdzane są kolizje, aktualizowane położenia przeszkód, generowane są brakujące pasy, przesuwana jest kamera oraz weryfikowany jest warunek przegranej wynikający z pozostania za ekranem.

Fragment 3. Inicjalizacja timera i zwiększanie trudności:

```

self.timer = QTimer()
self.timer.timeout.connect(self.update_game)
self.timer.start(16)

passed_rows = abs(self.player_world_row - self.start_world_row)
current_step = passed_rows // 5
if current_step > self.last_difficulty_step:
    self.difficulty += (current_step - self.last_difficulty_step) * 0.5
    self.apply_difficulty_to_lanes()

```

Ruch kamery jest niezależny od pojedynczego naciśnięcia klawisza i stale wymusza postęp. Rozwiązanie to nadaje grze charakter zręcznościowy: zatrzymanie postaci przez zbyt długi czas prowadzi do przegranej.

4.4. Obiekty ruchome i detekcja kolizji

Samochody pojawiają się na jezdniach i stanowią przeszkody śmiertelne. Kłody pojawiają się na rzekach i są platformami, które przemieszczają gracza wraz ze swoim ruchem. Kolizja jest sprawdzana w osi poziomej przez porównanie przedziałów zajmowanych przez gracza i obiekt.

Fragment 4. Zasada sprawdzania kontaktu z obiektem:

```

player_x1 = self.player_col * TILE_SIZE + 5
player_x2 = player_x1 + TILE_SIZE - 10

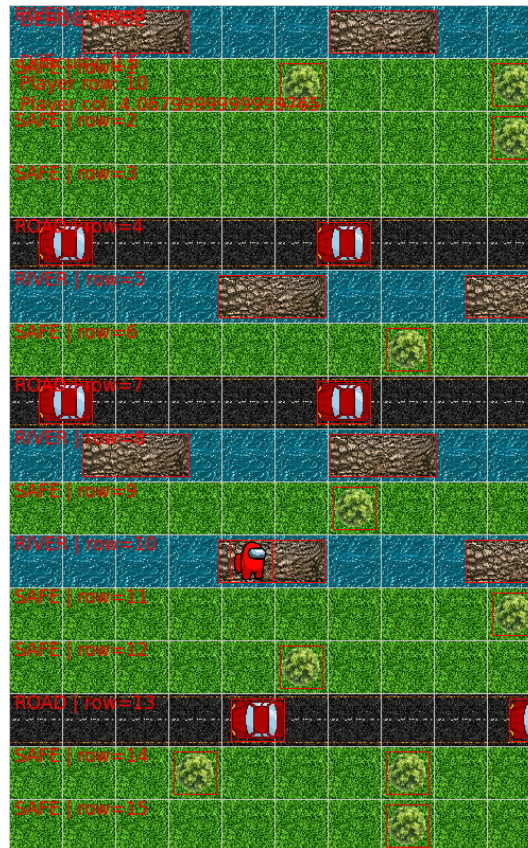
if not (player_x2 <= obs_x1 or player_x1 >= obs_x2):
    return obs # kłoda pod graczem lub trafienie samochodu

```

Na pasach trawiastych przeszkoda typu `tree` posiada atrybut `solid=True`. Metoda `can_move` blokuje wejście gracza na takie pole, natomiast dla drogi i rzeki ruch jest dopuszczony, a jego konsekwencje oceniają osobne metody kolizyjne.

4.5. Renderowanie i tryb diagnostyczny

Grafika jest rysowana w metodzie paintEvent za pomocą QPainter. Kolejno wyświetlane są tła pasów, siatka pól, przeszkody oraz postać gracza. Tekstury są ładowane z katalogu textures. Po wciśnięciu F1 program wyświetla tryb diagnostyczny zawierający bieżący poziom trudności, pozycję gracza, typy pasów i obrysy obiektów.



Rys. 3. Tryb diagnostyczny programu aktywowany klawiszem F1.

4.6. Zapis, odczyt oraz replay

Zapis gry jest realizowany w formacie JSON. Zapisywane są między innymi: pozycja gracza, trudność, status życia oraz wszystkie wygenerowane pasy wraz z obiektami. Dzięki temu po odczycie użytkownik otrzymuje planszę odpowiadającą zapisanej sytuacji. Funkcje są przypisane do klawiszy S (zapis) i L (odczyt).

Fragment 5. Reprezentatywny zakres danych zapisywanych w pliku JSON:

```
data = {
    "player_col": game.player_col,
    "player_world_row": game.player_world_row,
    "difficulty": game.difficulty,
    "is_alive": game.is_alive,
    "lanes": []
}
# Do lanes dołączane są parametry pasów i listy obiektów.
```

Mechanizm replay przechowuje ruchy wraz z numerem kroku zegara gry. Po wybraniu klawisza R przywracany jest zapamiętany stan początkowy, a następnie wykonywane są ruchy w

przypisanych im taktach. Logger ułatwia obserwowanie wygenerowanych pasów, zmian trudności oraz operacji zapisu i odczytu.

4.7. Rozbudowany interfejs graficzny i poziomy trudności

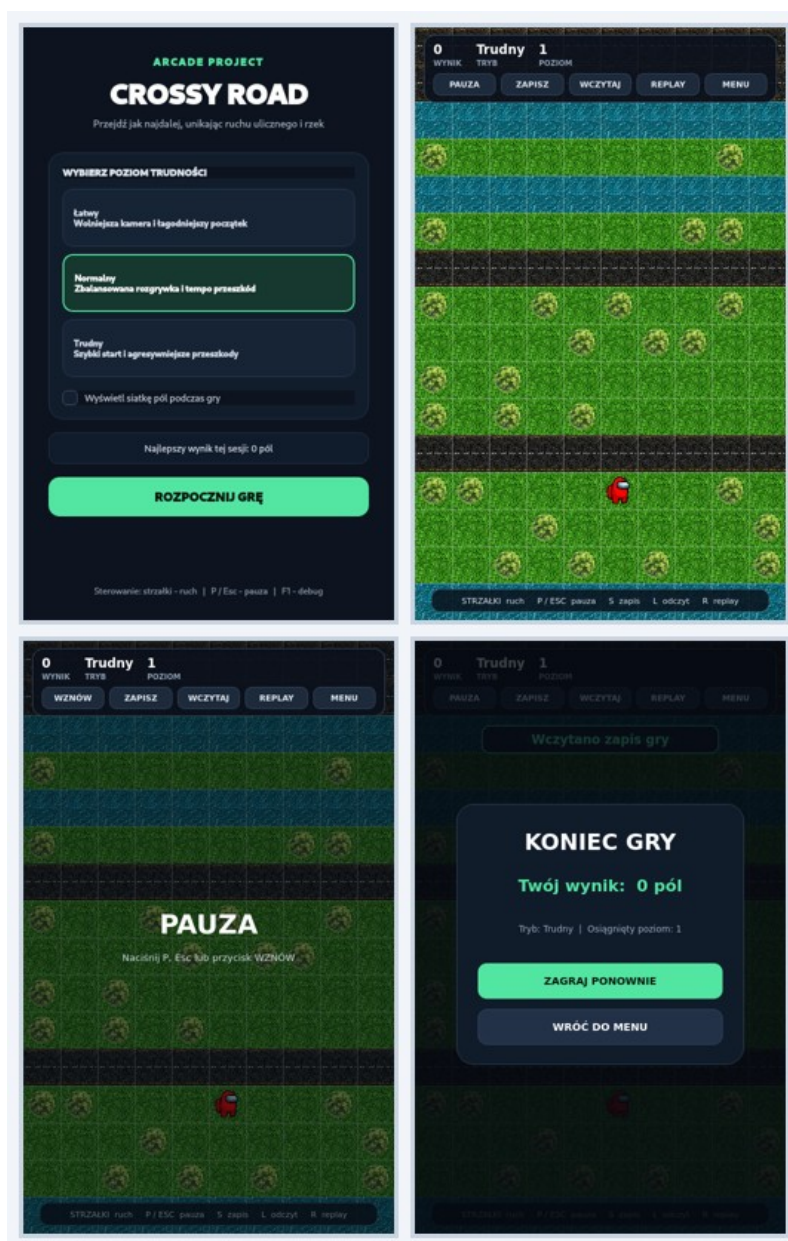
W nowej wersji zastosowano klasę MainWindow wykorzystującą QStackedWidget. Dzięki temu aplikacja przełącza się pomiędzy ekranem menu a właściwą rozgrywką bez otwierania nowych okien. MenuPage prezentuje nazwę gry, trzy warianty trudności, opcję siatki pól oraz najlepszy wynik uzyskany w bieżącej sesji.

Fragment 6. Presety wybierane w ekranie startowym:

```
PRESETS = {
    "easy": {"label": "Łatwy", "initial_difficulty": 0.15, "camera_speed": 0.20},
    "normal": {"label": "Normalny", "initial_difficulty": 0.50, "camera_speed": 0.30},
    "hard": {"label": "Trudny", "initial_difficulty": 1.35, "camera_speed": 0.43},
}
```

Wybrany preset przekazywany jest do obiektu Game przy rozpoczęciu gry. Tryb wpływa od pierwszej klatki na początkową wartość trudności oraz prędkość przesuwania kamery, dlatego wybór użytkownika ma realny wpływ na dynamikę rozgrywki, a nie wyłącznie na nazwę wyświetlaną w interfejsie.

W czasie gry u góry planszy znajduje się panel HUD zawierający wynik, wybrany tryb oraz poziom. Dostępne są przyciski Pauza, Zapisz, Wczytaj, Replay i Menu. Wciśnięcie P lub Esc zatrzymuje timer i wyświetla warstwę pauzy. Po przegranej pojawia się spójny wizualnie ekran końcowy z wynikiem oraz możliwością restartu lub powrotu do menu.



Rys. 4. Kolejne stany interfejsu: menu, rozgrywka z HUD-em, pauza oraz ekran końca gry.

5. Obsługa i uruchomienie aplikacji

Program wymaga interpretera Python oraz biblioteki PyQt5. Plik Pip Install.txt dostarczony z projektem wskazuje instalację zależności poleceniem `pip install PyQt5`. Po uruchomieniu pliku `main.py` wyświetlane jest menu główne. Użytkownik wybiera poziom trudności i opcjonalną siatkę pól, po czym rozpoczyna rozgrywkę przyciskiem **ROZPOCZNIJ GRĘ**.

Klawisz	Działanie
Strzałka w górę	Przejsie o jeden pas w kierunku postępu gry.
Strzałka w dół	Cofnięcie postaci o jeden pas.

Klawisz	Działanie
Strzałka w lewo / prawo	Zmiana kolumny, o ile pole nie jest zablokowane.
S	Zapis bieżącego stanu gry do pliku save.json.
L	Odczyt wcześniej zapisanego stanu gry.
R	Uruchomienie odtwarzania zarejestrowanych ruchów.
P / Esc	Wstrzymanie lub wznowienie rozgrywki.
Przyciski HUD	Pauza, zapis, odczyt, replay oraz powrót do menu głównego.
F1	Włączenie lub wyłączenie trybu diagnostycznego.

Przykładowy sposób uruchomienia:

Polecenia w terminalu:

```
pip install PyQt5
python main.py
```

6. Weryfikacja działania i osiągnięte wyniki

W ramach przygotowania sprawozdania przeprowadzono analizę kodu oraz test uruchomieniowy aplikacji w trybie renderowania bez wyświetlania okna na pulpicie. Zrewidowana wersja kodu przechodzi kontrolę składni modułów. Wygenerowane widoki interfejsu przedstawiają ekran menu, planszę z panelem HUD, warstwę pauzy i ekran końca gry. Zachowano dotychczasowe mechanizmy generowania świata, kolizji, zapisu, odczytu oraz trybu diagnostycznego.

Przeprowadzono również testy kontrolne kluczowych mechanizmów. Wyniki zestawiono poniżej:

Sprawdzany element	Sposób sprawdzenia	Wynik
Poprawność składni modułów	Kompilacja plików *.py przez moduł py_compile.	Pozytywny

Sprawdzany element	Sposób sprawdzenia	Wynik
Reguła generatora pasów	Wygenerowanie 200 pasów dla ustalonego ziarna i kontrola braku dwóch rzek z rzędu.	Pozytywny
Zapis i odczyt stanu	Zapis nowego stanu, modyfikacja pozycji, ponowny odczyt i porównanie pozycji oraz liczby pasów.	Pozytywny
Zwiększanie trudności	Symulacja przejścia pięciu wierszy od pozycji początkowej.	Pozytywny
Rejestracja ruchu replay	Wykonanie poprawnego ruchu i zapisanie go wraz z numerem taktu.	Pozytywny
Menu i wybór trybu	Analiza sygnału start_game oraz przekazywania presetu do obiektu Game.	Zaimplementowano
Pauza i koniec gry	Analiza zatrzymania timera i warstw interfejsu generowanych w Game.	Zaimplementowano
Ustawienia w zapisie	Kontrola pola show_grid i danych trybu zapisywanych w formacie JSON.	Zaimplementowano

Osiągnięty rezultat spełnia główny cel projektu: użytkownik otrzymuje działającą grę okienkową z dynamicznym światem, rosnącym poziomem trudności oraz dodatkowymi funkcjami zapisu i podglądu diagnostycznego. Zastosowana modularność umożliwia dalszą rozbudowę bez konieczności przebudowy całego programu.

7. Dyskusja wyników - zalety, wady i możliwości rozwoju

7.1. Zalety aplikacji

- Czytelny podział na moduły: generowanie świata, reprezentacja pasów, przeszkody, zapis, replay i logowanie są rozdzielone od głównej klasy okna.
- Proceduralne generowanie zapewnia zmienność rozgrywki, a jednocześnie zawiera regułę ograniczającą nadmiernie trudne układy dwóch rzek z rzędu.

- Skalowanie prędkości i interwałów generowania obiektów pozwala zwiększać trudność wraz z postępem użytkownika.
- Obsługa tekstur i prostego debug mode ułatwia prezentację projektu oraz wykrywanie błędów logicznych.
- Funkcja zapisu i odczytu rozszerza aplikację poza minimalną grę zręcznościową.
- Spójny graficznie interfejs z menu, HUD-em, pauzą i ekranem końca gry poprawia prezentację projektu.
- Trzy poziomy trudności pozwalają dopasować dynamikę rozgrywki do użytkownika.

7.2. Ograniczenia i wady zauważone w obecnej wersji

Ograniczenie	Znaczenie praktyczne	Proponowane usprawnienie
Replay nie zapisuje stanu generatora losowego.	Po odtworzeniu dłuższej sesji nowo generowane pasy mogą różnić się od pierwotnej rozgrywki.	Zapisać ziarno i stan generatora lub rejestrować komplet wygenerowanych zmian świata.
Zapis nie obejmuje wszystkich parametrów bieżącej sesji.	Nie są zachowywane m.in. precyzyjne położenie kamery, historia replay i stan generatora; odczyt nie zawsze jest identycznym wznowieniem.	Rozszerzyć schemat JSON i dodać wersjonowanie formatu zapisu.
Brak obsługi wyjątków przy odczycie pliku.	Uszkodzony lub brakujący save.json może przerwać działanie aplikacji.	Dodać walidację danych, obsługę FileNotFoundError i komunikaty użytkownika.
Brak trwałego rekordu pomiędzy uruchomieniami.	Najlepszy wynik działa tylko w bieżącej sesji aplikacji.	Zapisywać rekord w oddzielnym pliku konfiguracyjnym lub w JSON.
Parametry trybów są zapisane w kodzie.	Zmiana balansu wymaga edycji źródeł programu.	Przenieść konfigurację poziomów do pliku ustawień.

7.3. Kierunki dalszego rozwoju

Wersja zaktualizowana zawiera już punktację, rekord sesji, pauzę, restart i wybór poziomu trudności. Najbardziej wartościowym kolejnym rozwinięciem byłoby zapisanie rekordu pomiędzy uruchomieniami, dodanie konfiguracji sterowania i ekranu ustawień oraz poprawienie pełnej deterministyczności mechanizmu replay. Od strony jakości kodu wskazane jest wydzielenie logiki modelu gry od warstwy rysowania, dodanie automatycznych testów jednostkowych dla generatora i kolizji oraz dopracowanie deterministycznego replay.

8. Podsumowanie

W ramach projektu wykonano działającą aplikację typu Crossy Road w języku Python z biblioteką PyQt5. Program realizuje podstawowe wymagania gry zręcznościowej: dynamicznie tworzoną planszę, ruch postaci, ruchome przeszkody, wykrywanie kolizji, kamerę wymuszającą postęp oraz wzrost trudności. Dodatkowo zaimplementowano zapis i odczyt stanu, replay, logowanie zdarzeń, tryb diagnostyczny oraz rozbudowane GUI z menu startowym, wyborem trudności, panelem HUD, pauzą i ekranem końca gry.

Przyjęta struktura modułowa jest mocną stroną rozwiązania i stanowi dobrą podstawę do dalszej rozbudowy. Wykonane sprawdzenia potwierdziły działanie najważniejszych mechanizmów. Jednocześnie analiza wskazała konkretne miejsca wymagające dalszej poprawy, zwłaszcza w zakresie pełnej deterministyczności powtórek, odporności obsługi zapisu oraz trwałego zapisywania najlepszego wyniku.

Projekt można uznać za kompletną i atrakcyjnie zaprezentowaną aplikację okienkową, przedstawiający praktyczne wykorzystanie zdarzeń, timera, grafiki 2D, obiektowego modelu danych i serializacji stanu aplikacji.

Materiały źródłowe

1. Kod źródłowy projektu `Crossy_road_game_project` przekazany do opracowania sprawozdania: pliki Python, zasoby graficzne oraz plik `save.json`.
2. Dokumentacja funkcjonalności opracowana na podstawie analizy i uruchomienia dostarczonej aplikacji.